Acoris

Credit that proves itself.

A Technical Whitepaper

AI-negotiated DeFi lending on Creditcoin CC3 Testnet, priced only on financial history that has been cryptographically verified.

Version 1.0 · September 2026

Built for BUIDL CTC Fall 2026

`0x8cB3dDFF9e432D23e622Ff118A3fDDA192993Fb4`

Abstract

Acoris is a DeFi lending protocol in which loan terms are negotiated by two AI agents — a Borrower AI and a Lender AI — but priced and enforced by deterministic code. The model proposes; it never enforces. Terms are priced favorably only when backed by **evidence**: a real repayment, independently re-verified either cross-chain (via Attestcoin) or natively on Creditcoin. A self-reported claim with no backing proof is priced identically to having no claim at all — by construction, not by prompt. Accepted terms settle as a real transaction on `AcorisLoanRegistry`, a Solidity contract deployed on Creditcoin CC3 Testnet that escrows collateral, forwards principal, computes interest, and returns collateral on repayment. This document describes the protocol's evidence model, its negotiation and pricing mechanics, its on-chain settlement layer, its security assumptions, and defines every term used throughout the system.

1. The Problem

DeFi lending protocols generally treat every wallet as an anonymous, contextless address. A wallet's real repayment history — on the protocol in question, or on any other — carries no weight in how the next loan is priced. Borrowers rebuild trust from nothing on every new protocol, no matter how much genuine repayment history they hold elsewhere. And because there is no cheap way for a lender to check a claim, self-reported history ("I always repay on time") is exactly as easy to say as it is impossible to verify without doing the work yourself.

Acoris addresses this directly: it gives a borrower a way to *prove* a real repayment rather than merely claim one, and it gives a lender terms that are priced against that proof — not against a story.

2. Design Principles

Four principles govern every feature described in this document, and are referenced throughout rather than restated each time:

I. **Evidence-only pricing.** Only genuinely verified financial history is ever allowed to influence loan terms. An unverified claim is priced identically to no claim at all.

II. **The model proposes, code enforces.** Both AI agents may propose any numbers they like; a separate, deterministic layer clamps every proposal to hard limits before it becomes an official round. The model is never trusted to enforce its own limits.

III. Nothing is fabricated. No invented credit score, no fabricated transaction, no simulated confirmation, no placeholder activity. Where a fact cannot be honestly derived, the interface says so explicitly rather than approximating it.

IV. The chain is the source of truth. Once an agreement exists on `AcorisLoanRegistry`, its real on-chain state — never a cached copy, never client-supplied data — is what every view of it reads.

3. Protocol Overview

Acoris runs as a loop with four stages, each producing evidence for the next cycle:

Verify → A borrower proves a real repayment.

Negotiate → Borrower AI and Lender AI exchange structured offers.

Fund → Accepted terms settle as a real on-chain agreement.

Repay → Repayment returns collateral and becomes evidence for the next cycle.

Two independent actors are involved in every deal: a **Borrower**, who requests credit and eventually repays it, and a **Lender**, who reviews and funds it. Nothing in Acoris requires them to be the same person or to use the same device — they act through independently connected wallets, and the on-chain contract itself is the only thing that has to agree with both of them.

4. Evidence

Evidence is the central concept in Acoris: it is the only thing pricing is ever allowed to depend on. This section defines exactly what evidence is, the forms it can take, how it is represented internally, and how it is produced.

4.1 Financial Profile Status

Every credit decision in Acoris consumes a **Verified Financial Profile**, which always has exactly one of three statuses:

STATUS	MEANING
`not-available`	No evidence was submitted or attempted. Priced as an unknown borrower — not treated as bad credit, simply absent.
`unverified`	A self-reported claim was made but carries no cryptographic backing. Priced *exactly* like `not-available` — an unverified claim never earns better terms, by construction.

`verified`	Real evidence was independently re-derived by the server from a cryptographic proof or an on-chain event log. The only status that can improve pricing.

4.2 Evidence Modes

A user selects how to supply evidence through one of four modes, each mapped to a specific server-side resolution path. Critically, the server never trusts a client-supplied "verified" result — it only ever accepts raw inputs (a transaction hash, a wallet address) and independently re-derives the evidence itself.

MODE	INPUT	RESOLVES TO
`none`	—	`not-available`
`unverified`	A free-text claim (optional)	`unverified`, claim carried through for display only, never priced
`verify`	1–10 Sepolia transaction hashes	Each hash independently re-verified via the Attestcoin pipeline (§4.4); only genuinely successful verifications count
`onchain`	A borrower wallet address	That address's real `AcorisLoanRegistry` event history, read directly from CC3 Testnet (§4.5)

4.3 The Verified Financial Profile

When evidence resolves to `verified`, the profile carries the following fields — every one of them a direct function of real evidence, never estimated:

FIELD	MEANING
`verifiedRepaymentCount`	Total number of verified repayment events, successful or failed.
`verifiedRepaymentVolume`	Total repaid value across all verified evidence, in wei.
`successfulRepaymentCount`	Verified events where the repayment actually succeeded.
`failedRepaymentCount`	Verified events where the repayment failed (e.g. a default).
`onTimeRepaymentRate`	Fraction of repayments made by their due date, 0–1. `null` when no evidence in the set carries a determinable due date (true of every Sepolia-via-Attestcoin entry, since that evidence shape carries no due-date) — never invented from evidence that doesn't support it.
`mostRecentVerifiedActivity`	A description of the latest verified event (e.g. "Sepolia block 9123456").
`sourceChains`	Every chain this evidence was verified against.
`verificationEvidence`	The full list of individual evidence records backing this profile (see §4.3.1).

4.3.1 Verification Evidence Record

Each individual proof underlying a profile is represented as a record carrying: the source chain, the transaction hash, the block height, the decoded repayment amount (wei), a success/failure status, and an `onTime` boolean that is `null` whenever due-date derivation genuinely is not possible from that evidence source rather than guessed at.

4.4 Cross-Chain Verification — Attestcoin

The `verify` evidence mode proves a repayment that happened on a different chain (Sepolia) using Creditcoin's Attestcoin / USC verification pipeline. The pipeline runs six real stages for every transaction hash submitted, with no shortcut and no fallback that fabricates a result on failure:

1. List supported source chains live from the ChainInfo precompile and resolve Sepolia's chain key dynamically — never hardcoded.
2. Fetch the source transaction and its receipt directly from Sepolia.
3. Check whether the source block's height is attested on Creditcoin CC3 (via the ChainInfoProvider's continuity bounds).
4. Fetch a Merkle and continuity proof from the hosted CC3 Proof Builder service.
5. Verify that proof on-chain via the BlockProver precompile.
6. Decode only the specific fields Acoris needs from the now-verified transaction bytes into a structured fact.

Any failure at a network-bound stage is surfaced honestly as a failed verification with the real stage and error message attached — never silently treated as success.

4.5 Native On-Chain Verification

The `onchain` evidence mode needs no cross-chain proof at all: because `AcorisLoanRegistry` lives on the same chain the negotiation engine runs against, its own event log is already the trustless source of truth. Given a borrower address, Acoris reads every `AgreementProposed`, `FundLoan`, `RepayLoan`, `AgreementDefaulted`, and `AgreementCancelled` event for that address directly from CC3 Testnet and reconstructs a full **Loan Timeline** per agreement — proposed-at, funded-at, a real due date (funded-at plus duration), repaid-at, and a genuine `onTime` boolean computed from the loan's actual timestamps, not assumed.

4.6 Risk Discount

Verified evidence is turned into pricing through exactly one formula — the same formula both the negotiation engine and the standalone underwriting view (§7) read from, never re-derived differently in two places:

```
countScore = min(verifiedRepaymentCount ÷ 5, 1)
reliabilityScore = 1 − (failedRepaymentCount ÷ totalRepayments)
riskDiscount = countScore × reliabilityScore, clamped to [0, 1]
```

A `riskDiscount` of 0 means baseline pricing; 1 means a lender's best-case pricing. `countScore` saturates at five verified repayments — more history beyond that earns no further discount. Both `countScore` and `reliabilityScore` are `null`, not zero, when there is no verified evidence to compute them from, so a caller can distinguish "no evidence" from "evidence exists and scored zero."

5. The Negotiation Engine

Two AI agents negotiate every deal: a **Borrower AI**, acting on the borrower's stated request and hard limits, and a **Lender AI**, pricing risk against the borrower's verified financial profile. Both are backed by Claude (Anthropic API) and produce a structured **Negotiation Round** on each turn — never free-form text.

5.1 Negotiation Rounds

Each round is a machine-readable record: which agent acted, one of four actions, concrete proposed terms, a reasoning string, and whether deterministic clamping altered the proposal before it was recorded.

ACTION	MEANING
`OFFER`	An opening proposal.
`COUNTER`	A revised proposal in response to the other side.
`ACCEPT`	The current terms are accepted — negotiation ends successfully.
`REJECT`	The negotiation ends without agreement.

A negotiation runs for at most eight rounds. If no agreement is reached by then, it ends as `no-agreement` — never coerced into a fabricated acceptance.

5.2 Constraints & Deterministic Enforcement

Neither agent is trusted to respect its own limits. Instead, two constraint sets are derived independently of the AI and applied to every proposal before it counts:

CONSTRAINT	FIELD	MEANING
Borrower	`maxApr`	Never accepts an APR above this.

	`minAmount`	Never accepts less principal than this.
	`maxCollateral`	Never posts more collateral than this.
	`minDurationDays / maxDurationDays`	Acceptable duration range.
Lender	`minApr`	Never accepts an APR below this — derived from the lender's risk policy and the borrower's `riskDiscount` (§4.6).
	`maxAmount`	Never lends more than this.
	`minCollateralRatio`	Minimum collateral / principal ratio required.
	`maxDurationDays`	Longest duration this lender will accept.

Whenever a proposal from either agent violates its counterparty's constraints, the numbers are **clamped** — adjusted to the nearest valid value — before being recorded as an official round, and the round is flagged `wasClamped` so the UI can show that a limit was actually enforced, not simply respected by chance.

6. The Lender Marketplace

Rather than negotiate against one fixed lender policy, a borrower can request offers from three named **Lender Personas** at once — each a genuinely distinct risk policy, each pricing the same request with its own real, independent AI call:

PERSONA	STYLE	BASE APR FLOOR	BASE COLLATERAL RATIO
Lender Alpha	Conservative	8.0%	1.55
Lender Beta	Balanced	9.0%	1.50
Lender Gamma	Aggressive	9.5%	1.35

The tradeoff is real, not cosmetic: Alpha demands more collateral in exchange for a lower rate; Gamma accepts less collateral by pricing that risk into a higher rate. Lender Beta's policy is defined to match Acoris's own default single-lender policy exactly, so choosing not to use the marketplace changes nothing about how a direct negotiation is priced.

Once all three offers resolve, the best *affordable* one — the lowest APR among offers whose required collateral the borrower can actually meet — is selected by a plain, deterministic rule, never an invented "AI judgment" score. If none are affordable, the marketplace reports that honestly rather than choosing anyway.

7. AI Credit Underwriter

The underwriting view answers "how would a lender price me?" using the exact formula described in §4.6 and §5.2 — and, deliberately, makes no AI call at all. It narrates the same real `riskDiscount`, `countScore`, and `reliabilityScore` that already govern real pricing, in plain language, and works even when no AI provider key is configured. There is no invented "confidence 87%" anywhere in this view: if a number cannot be honestly derived from real evidence, it is left out rather than approximated.

8. Credit Improvement Simulator

This view answers "how can I get better terms?" by feeding a concrete, explicitly stated hypothetical — for example, "five verified repayments instead of two, same reliability" — through the same unmodified pricing function described in §4.6, and reporting the real resulting constraints. Every projected number is produced by the real formula against a real hypothetical input; none is a promised future rate, and a suggestion that has no honest hypothetical to project from (an unverified claim, for instance) is shown with no projected numbers at all rather than a guessed one.

9. On-Chain Settlement — AcorisLoanRegistry

`AcorisLoanRegistry` is a single Solidity contract, deployed once, used for every deal. Principal and collateral are native CTC (Creditcoin's testnet gas token) — no ERC-20 approval flow is needed. Each agreement is keyed by a `bytes32 loanHash`, derived deterministically from the negotiation's own id, and moves through exactly one of the following lifecycles:

Proposed → **Cancelled** (borrower withdraws, collateral refunded)
Proposed → **Funded** → **Repaid** (lender receives principal + interest; borrower's collateral returned)
Proposed → **Funded** → **Defaulted** (past due date; lender seizes collateral)

9.1 Contract Functions

FUNCTION	CALLER	EFFECT
`proposeAgreement(loanHash, lender, principal, aprBps, durationSeconds)`	Borrower	Escrows `msg.value` as collateral; status → `Proposed`.
`cancelProposal(loanHash)`	Borrower	Refunds collateral; status → `Cancelled`. Only while still `Proposed`.
`fundAgreement(loanHash)`	Lender	`msg.value` must equal principal exactly; forwards principal to the borrower; status → `Funded`.

`repay(loanHash)`	Borrower	`msg.value` must equal `repaymentAmount` exactly; pays the lender, returns collateral; status → `Repaid`.
`markDefaulted(loanHash)`	Lender	Only after the due date has passed; seizes collateral; status → `Defaulted`.
`repaymentAmount(loanHash)`	Anyone (view)	Returns principal + interest owed right now.

9.2 Interest

Interest is simple (non-compounding) over the full agreed duration:

```
interest = principal × aprBps × durationSeconds ÷ (365 days × 10000)
owed = principal + interest
```

9.3 Access Control

Every state-changing function checks `msg.sender` against the exact address recorded at proposal time. `fundAgreement` reverts with `NotLender()` if called by any address other than the one the borrower named as lender — this is a hard, on-chain guarantee, not a client-side convenience. Viewing an agreement's terms carries no such restriction, since contract state on a public chain is inherently public; only *funding* it is cryptographically restricted.

10. Discovery — the Agreement Link

Proposing an agreement writes it to the contract; nothing is emailed, pushed, or otherwise sent to the counterparty automatically — there is no backend capable of that. Instead, a borrower is given a shareable link (`/agreement?loanHash=...`) the moment a proposal confirms, which they send to their lender out of band. Opening that link resolves the `loanHash` (or, alternately, the negotiation id it was derived from) and reads the agreement directly from the chain — no negotiation history, no server-side record, needs to exist for this to work.

11. Portfolio & Notifications

A connected wallet's dashboard reads two independent views of the same contract: its own history as a **borrower**, and every agreement where it has been named as **lender** by someone else. The second view is filtered to agreements still awaiting funding and surfaced as a live count badge on the Portfolio link in the navigation, visible from any page in the app — the closest thing Acoris has to a notification,

built entirely from a periodic read against the contract's own `AgreementProposed` event log rather than any push infrastructure.

12. Security & Trust Model

- The server never accepts a client-supplied verification result — only raw inputs it independently re-derives evidence from.
- Funding rights are enforced on-chain by address match, not merely hidden in the UI; the contract itself reverts an unauthorized funding attempt.
- No user data, credentials, or off-chain records are stored anywhere; every fact shown is either read live from a chain or produced by a real cryptographic verification at request time.
- Both AI agents propose only — every hard limit is enforced by deterministic code that never delegates enforcement to the model.
- Acoris runs on Creditcoin CC3 **Testnet**. No real funds are involved anywhere in the deployment described in this document.

13. Technology Stack

Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS v4
Chain interaction	ethers v6, Creditcoin CC3 Testnet (chain id 102031)
Negotiation agents	Claude (Anthropic API) — structured output, proposes only
Smart contract	Solidity, Hardhat 3, real-EVM Mocha test suite
Cross-chain verification	Attestcoin / `@gluwa/usc-sdk`, BlockProver precompile
Request validation	Zod, at every API boundary

14. Verification Record

As of this document's publication, the following has been confirmed against the real deployment at `0x8cB3dDFF9e432D23e622Ff118A3fDDA192993Fb4` on Creditcoin CC3 Testnet (chain 102031):

- A complete propose → fund → repay cycle, run live with real transactions.
- The same cycle repeated with two genuinely separate wallets — an independent borrower and lender, each funded via the CC3 faucet — including sharing the `/agreement` link between them.
- 135 unit tests covering pricing, negotiation, evidence resolution, and on-chain event reconstruction; 20 real-EVM contract tests covering every state transition in §9; 3 end-to-end browser test suites.

- Three real defects found and fixed by direct reproduction during live testing, not guessed at — documented in full in this repository's technical docs.

The one lifecycle path not yet exercised live is `markDefaulted`'s post-due path, which requires deliberately letting a real loan go unpaid past its due date; it is covered by dedicated contract tests in the interim.

Glossary of Terms

Agreement

A single loan deal recorded on `AcorisLoanRegistry`, keyed by its `loanHash`.

AcorisLoanRegistry

The Solidity contract that executes every agreement's lifecycle on-chain (§9).

Agreement Status

One of `None`, `Proposed`, `Funded`, `Repaid`, `Defaulted`, `Cancelled` — the contract's own record of where a deal stands.

APR (aprBps)

Annual percentage rate, stored on-chain in basis points (900 = 9.00%).

Attestcoin

Creditcoin's cross-chain verification system (USC), used to prove a Sepolia repayment happened, without trusting a claim about it (§4.4).

Borrower AI

The negotiation agent acting on the borrower's stated request and hard limits (§5).

Clamping

The deterministic process of adjusting a proposed term to the nearest value that satisfies a hard constraint, before it is recorded as an official round (§5.2).

Collateral

Value the borrower posts at proposal time, escrowed by the contract and returned on repayment (or seized on default).

Collateral Ratio

Collateral value divided by principal — the minimum a lender will accept is `minCollateralRatio`.

Constraints

The deterministic hard limits — separate for borrower and lender — that neither AI agent is trusted to enforce itself (§5.2).

Evidence

A borrower's proven financial history, in one of three states: `not-available`, `unverified`, or `verified` (§4).

Financial Profile

See Verified Financial Profile.

Lender AI

The negotiation agent pricing risk against the borrower's verified financial profile (§5).

Lender Marketplace

The feature offering a borrower simultaneous, independently priced offers from three named lender personas (§6).

Lender Persona

One of three named, distinctly-risk-postured lender policies — Alpha (Conservative), Beta (Balanced), Gamma (Aggressive) (§6).

Loan Hash (loanHash)

The unique `bytes32` identifier for one agreement, derived deterministically from its negotiation id.

Loan Timeline

A reconstructed record of one agreement's real on-chain history — proposed-at, funded-at, due date, repaid-at — built from its actual event log (§4.5).

Negotiation Round

One structured turn in a negotiation: an agent, an action, proposed terms, reasoning, and whether clamping altered it (§5.1).

Principal

The loan amount requested and, once funded, transferred to the borrower.

Repayment Amount

Principal plus interest owed at the time of repayment, computed live by the contract itself (§9.2).

Risk Discount

A value from 0 to 1 expressing how far a borrower's verified evidence moves their pricing from a lender's baseline toward its best case (§4.6).

Verified Financial Profile

The structured record of a borrower's evidence, consumed by every pricing decision in the protocol (§4.3).

Appendix — Links

Repository	`github.com/TeevincsCrypt/Acoris`
Contract (CC3 Testnet)	`0x8cB3dDFF9e432D23e622Ff118A3fDDA192993Fb4`
Explorer	`creditcoin-testnet.blockscout.com/address/0x8cB3dDFF9e432D23e622Ff118A3fDDA192993Fb4`
Technical documentation	`/docs in the repository`

Every claim in this document is backed by something checkable in the repository above — source code, tests, or the deployed contract itself. Nothing here is asserted that the code does not already demonstrate.